Instructor Notes

Pedro Pathing 3 for FIRST Tech Challenge · Deck B: Pedro in Code
Companion to **FTC_PedroPathing_DeckB_Practical.pdf** (16 slides).
Follows Deck A (Concepts).

The lesson in one line. Seven steps from easy to harder: draw a path, run one path, chain paths, choose the heading, mirror for the other alliance, drive in TeleOp, check your work. Students **type and run** each step on the robot or a laptop, not just read it.

Timing. About 95 minutes in two sessions: slides 1–9 (about 55 min), slides 10–16 (about 40 min). Each entry gives a suggested time, what to say, a question to ask, and what to watch out for. Answers are in [brackets]. It builds on Deck A: students should already know Pose, heading, PoseFactory, localizer, follower and Foresight.

Where the code comes from. Every code block is copied or trimmed from the official Pedro 3 docs (pedropathing.com/docs), except two things I label on the slides and below: (1) the **state machine** on slides 7–8 is our team's pattern, built only from calls the docs show (`follow, isBusy, update, atParametricEnd`); (2) `follower.follow(path)` is how the docs say a path can be followed directly, and PathLab uses it. The docs' own examples use the Ivy command library; we chose state machines, so no slide shows Ivy.

What I could not check. I have **not** compiled or run any of this code, and I have not read the PathLab source. I could not operate the Pedro Visualizer in this session, so slide 4 describes the idea, not exact buttons. I do not know our robot's Constants or whether Foresight is switched on in them. **Type the slide 5–7 code into a project and build it before class.** Fix anything my transcription got wrong.

Before class

Item	Why
A project that already passes the Tuning Checklist	Deck B assumes the Tests on slide 3 pass. If the robot strafes during a Line Test or the pods are wrong, no code on these slides will fix it.
A laptop with Android Studio and the Visualizer open	Slides 4 and 5–7 are meant to be typed and run live.
FTC Dashboard open at `http://192.168.43.1:8080/dash`	Used in the demo strips on slides 4, 10 and 13.
A tape X on the floor at (24, 24)	Start spot for the PathLab tests and the first Auto.

Slide by slide

1 Title

1 min.

Say: Deck A told you what Pedro does. Deck B is where you make it do it. Each step is something you can type and run.

---

2 Seven steps, easy to harder

3 min.

Say: Read the seven steps. Then the tools: the Visualizer to draw, the Dashboard to see, telemetry to print, the Tuning Checklist to get the robot ready. Say it plainly: if the robot is not tuned, the code still looks right and the robot still drives wrong.

Ask the class: Which step do you think is hardest? [Any answer. Many say 3 or 5.]

Watch out: The Tuning Checklist is our team's document and is not in this deck; the next slide is only a map of it. Do not try to teach tuning here. The Visualizer is at `visualizer.pedropathing.com`.

---

3 The big picture: from Constants to a tested robot

6 min. Draw it on the board as you go.

Say: Walk left to right. **Install** Pedro. Then tune three things, in the order the docs give: the **drivetrain** (AutoTune finds the motor names and directions), the **localizer** (we use the Pinpoint: set its offsets and pod directions), and **Foresight** (AutoTune measures top speeds, braking and gains). Each one produces a config that goes into one file, `Constants.java`. The `create` method in that file builds the Follower from those three configs. Then run the **Tests** procedure in AutoTune: seven tests, from simple to demanding. Only then do you write paths.

Ask the class: Which test would show that the localizer is wrong? [Pose Test, or the Localization Test: the position should start at (0, 0), forward should increase x, and a left strafe should increase y.] Which would show Foresight is not tuned? [Hold Test, Line Test.]

Watch out: Everything on this slide comes from the docs' Tuning, Mecanum, Localization, Foresight, Constants and Test pages. I did **not** read the Pinpoint's own tuning page, so the slide says only what the Constants page shows: offsets and pod directions. The Test page gives no pass or fail criteria for most tests, so I did not invent any; its one example is the Hold Test (push the robot and watch it correct). The Foresight page lists eleven AutoTune steps; I left them off the slide for simplicity. The team's Tuning Checklist is the working document: **compare it with this slide**, I have not seen it. Two open items on our robot, the Y-pod offset sign and the strafing in the Line Test, belong to this stage; do not call them solved.

---

4 Draw the path before you write the code

8 min. Live: open the Visualizer and draw the Test 3 curve.

Say: Open visualizer.pedropathing.com. Put the start at (24, 24), add a curve with a control point at (60, 24), end at (72, 60). Show the numbers and the exported Java. Read it with the class: they can now read every line.

Back to the demo: This is the same curve as Test 3 in the demo. Ask: where would the Dashboard show it?

Ask the class: Which number is the heading? [The third one in a Pose.] What would you change to make the curve bend more? [Move the control point.]

Watch out: I could not operate the Visualizer, so the six steps are generic. Earlier research showed the export uses `Paths.line(...).linear(start, end)` and PoseFactory; check this when you open it. The version I saw was 1.5.0. Buttons will differ.

---

5 Poses first, then a path between them

6 min. Students type it.

Say: Type it with them. The import line lets us write `line()` without `Paths.`. The factory is in degrees. `p.of(24, 24, 0)` is x, y, heading. Then the path method. `.linear(startPose, park)` turns the heading smoothly from the start's heading to the end's. If you add no heading rule, the robot faces the way it travels (docs).

Ask the class: What is the robot's heading at the start? [0°, facing right.] And at park? [90°, facing up.]

Watch out: The docs' example is exactly this: start (24, 24, 0), park (48, 48, 90). Put poses and paths at the top of the class, not in `loop()`. The imports Android Studio offers are PoseFactory, Pose, Path, and later Follower and Constants.

---

6 Three methods, three jobs

8 min. Students run it on the robot or in the simulator if you have one.

Say: Walk the three methods. `init()` builds the follower and says where the robot starts. `start()` begins the path once. `loop()` updates the follower every pass. Then press PLAY and watch it drive.

Back to the demo: PathLab Test 1 is this idea: one line, heading kept constant. Press A on it and point at `update()` in the loop.

Ask the class: What happens if we delete `follower.update()`? [The robot does not move; Pedro stops knowing where it is. Try it and show them.]

Watch out: The docs start the path with Ivy (`schedule(follow(follower, park()))`). I replaced that with `follower.follow(parkPath())`, which the docs say is possible and PathLab uses. I have not run my version. Also, `setPose` must match where the robot really sits.

---

7 One step at a time: a state machine

12 min. Students type it with two paths.

Say: The robot can only do one thing at a time, and `loop()` runs over and over. So we keep a note of which step we are on. Each pass: update the follower, look at the step, and if the path is finished, start the next one and change the step. Point at the diagram: the arrows are the `!follower.isBusy()` check.

Back to the demo: Run PathLab and watch the Message line step through READY, DRIVING_PATH, PAUSING_AT_ENDPOINT and RETURNING.

Ask the class: Where does the code say 'wait'? [Nowhere. It checks each pass and moves on.] How would you add a third path? [A new step, a new `case`.]

Watch out: This pattern is ours, not the docs'. The docs recommend a command library (Ivy) over a hand-built state machine; our team chose state machines, which is a fair choice for an Auto with a few paths, but tell students the other way exists. The code is a sketch: add your mechanism steps and a `DONE` action. PathLab's `Step` enum (READY, DRIVING_PATH, PAUSING_AT_ENDPOINT, RETURNING, STOPPED) is the real example; I have not read its source.

---

8 Is the path finished?

4 min.

Say: Two ways to ask 'are we there?'. `!isBusy()` means done and stopped at the target. `atParametricEnd()` means the robot reached the end of the path, even if it has not stopped (docs, Follow States). Either check goes in `loop()` and runs every pass.

Ask the class: Why not `while (follower.isBusy()) {}`? [The loop stops, update() stops, and the robot stops knowing where it is.]

Watch out: The docs also describe a looser check for faster autos (low tangential velocity and `distanceToEndpoint()` under 4). Mention it exists; do not teach it. The PathLab notes flagged `isBusy()` staying true as an open risk, so `atParametricEnd()` is a good thing to know.

---

9 Choose how the robot faces

6 min.

Say: Four rules, one line each. Tangent: faces the way it travels. Linear: turns smoothly. Constant: never turns. Facing point: always looks at one spot. The last column ties each to a PathLab test they have already seen. The piecewise block is optional.

Back to the demo: Map the table to Tests 1, 2, 3 and 7. Which was the facing-a-point test? [Test 7.] Ask them what they saw.

Ask the class: Which rule would you use to drive backwards to a wall without turning around? [Constant, with the start heading.] Which for a robot that must always aim at a goal? [Facing point.]

Watch out: These forms are from the docs' Interpolation page. Pass a **Pose**: its heading uses your factory's units. A bare number is read as **radians** (the page says radians and never mentions degrees). Copy the piecewise block from the docs, I shortened the names. PathLab also uses `reverseTangent()` and `Paths.through()` in tests 4 and 6; I did **not** find them on the Interpolation page, so they are not on this slide.

---

10 Write once, mirror for red

6 min. Students try it.

Say: Make a second factory that mirrors the first about the middle of the field, x = 70.75. Use it to make the red start. Same numbers in, mirrored poses out: (30, 30) becomes (111.5, 30).

Back to the demo: Edit the PathLab points on the Dashboard to mirrored x values. Dashboard edits are lost at power-off.

Ask the class: A blue Pose has x = 20. What is the red x? [141.5 − 20 = 121.5.]

Watch out: **The docs do not say what mirroring does to the heading.** The slide says so on purpose. Make them print the pose and check the field in the Visualizer. The 141.5 inch field is what Pedro's Visualizer, PoseFactory and PathLab use; Pedro's Coordinates page example implies 144, which I flagged earlier and have not resolved. I did not run `red.of(30, 30, 0)` to confirm 111.5.

---

11 The same follower can drive with sticks

6 min.

Say: The follower is not just for autonomous. In TeleOp, `follower.manual(powers)` drives with the sticks, and `update()` still runs every loop. Field-centric uses the robot's heading from the pose, so the sticks mean the field's up, not the robot's front.

Ask the class: If the localizer is wrong, what happens to field-centric driving? [The sticks point the wrong way. This is why the pose matters in TeleOp too.]

Watch out: All three forms come from the docs' TeleOp Usage page: `follower.manual(forward, lateral, turn)`, `ManualDrive.fieldCentric` and `ManualDrive.driveOrHold`. Imports: `com.pedropathing.drivetrain.DrivePowers` and `com.pedropathing.follower.ManualDrive`. Students often ask if field-centric is better; it is a team choice, not a Pedro rule.

---

12 Hand the pose from Auto to TeleOp

5 min.

Say: Auto ends somewhere. Save that pose in a shared place when the Auto stops. In TeleOp's `start()`, load it with `setPose` and update once. Now field-centric is correct from the first press.

Ask the class: What goes wrong if someone nudges the robot between Auto and TeleOp? [The stored pose is wrong. Reset it by hand, for example from a known corner.]

Watch out: This is the docs' OpModeStorage pattern, shortened. The docs also show relocalizing on a button press with a known corner pose. If TeleOp starts on its own with no Auto, the default stored pose is (0, 0, 0), which is almost never right.

---

13 Print what Pedro thinks

6 min. Students add telemetry to their Auto.

Say: Add four lines. Where Pedro thinks the robot is, what mode it is in. Then stand next to the robot and compare with the floor. Pose heading is in radians, so convert it. The table lists what else you can ask: how far along the path, and the debug maps for the drivetrain, algorithm and localizer.

Back to the demo: The Dashboard field view is where students saw the planned path and the robot together in the demo.

Ask the class: The robot reports x = 48 but a tape measure says 51. Which half of the box (Deck A, slide 8) is wrong? [The localizer.]

Watch out: The telemetry lines are the docs' own. `follower.debug().drivetrain().toString()` is from the Debugging page; the docs also suggest filming those strings when asking for help. Logging (the `FollowerLog`) is on a separate Logging reference page, not covered here.

---

14 When it goes wrong, check these first

6 min.

Say: Go row by row, symptom then check. Make them say the check before you reveal it. The last row matters: if the robot drives the wrong shape or strafes, the answer is usually in tuning, not in the code on these slides.

Ask the class: A student says the Auto works in the pit but not on the field. What do you check first? [Where the robot was placed versus `setPose`.]

Watch out: Every row comes from a docs page or from Deck A, but the order and wording are mine. The strafing during the Line Test and the Y-pod offset sign are open questions on our own robot; do **not** present them as solved.

---

15 Twelve calls you now own

3 min.

Say: Cover the right-hand column and quiz. Say each call out loud.

---

16 Check yourself

8 min.

Say: Six questions. Pairs discuss, then answer aloud. Answers: [1. `init()` runs once; `loop()` repeats, and the follower must update on every pass. 2. `!follower.isBusy()` (or `atParametricEnd()` for a looser check). 3. It freezes the loop, so `update()` never runs and the follower never moves. 4. Pedro thinks the robot is at (24, 24), so the whole path is offset by about 6 inches in x and 6 in y. 5. `facingPoint`. 6. Print the red start pose, check the heading, and view the path in the Visualizer.]

Sources

Pedro Pathing 3 docs: Installation, Tuning, Mecanum, Localization, Foresight, Constants, Test, Setting Up Your Autonomous, Creating a Pose, Creating a Path, Following a Path, Autonomous Usage, TeleOp Usage, Follow States, Debugging, Modifiers, and the Interpolation reference (pedropathing.com/docs/pathing). PathLab project documents (Path Tests User Guide, HLD, LLD). Deck A v4. Not used: Ivy, Pedro 2 material, the Pinpoint tuning page.